Implementation Plan: SODA-Based Adaptive Bitrate Optimization (Course Project)

Project Goal

The primary goal of this project is to reproduce Adaptive Bitrate (ABR) streaming behavior and demonstrate bitrate oscillation issues under fluctuating network conditions. We will implement the SODA (Smoothness Optimized Dynamic Adaptive) controller, which:

- Minimizes unnecessary bitrate switches using a highly efficient approximate solver.
- Maintains high visual quality while drastically reducing rebuffering and quality jumps.

Finally, we will compare SODA with a baseline robustMPC algorithm using the following Quality of Experience (QoE) metrics:

- **Average Bitrate / Utility**
- **Rebuffering Ratio**
- **Smoothness** (Bitrate Switching Rate)

System Architecture

The system follows a linear simulation flow: **Network Trace ➔ ABR Simulator ➔ ABR Algorithm** (robustMPC / SODA) **➔ QoE Metrics**

- **Network Traces:** Simulate real-world bandwidth variations.
- **ABR Simulator:** Models chunk-based video streaming and buffer dynamics.
- **ABR Algorithm:** Dynamically selects the next optimal bitrate.
- **Output Metrics:** Evaluates algorithm performance using standard QoE formulas.

Implementation Steps

Step 1: Simulation Setup

We will configure a controlled simulation environment using Python.

- **Engine:** Open-source Python Sabre simulator.
- **Chunk Duration:** 2 seconds.

- **Bitrate Levels:** [1.5, 4.0, 7.5, 12.0, 24.0, 60.0] Mbps (YouTube 4K recommended ladder).
- **Constraints:** Maximum buffer capacity of 20 seconds (downloads automatically pause if the buffer is full).

Step 2: Data Pipeline

- **Datasets:** We will utilize the Puffer Dataset (Jan–Jun 2023) and 4G/5G Irish mobile traces.
- **Preparation:** Filter all raw network traces into exact 10-minute sessions.
- **Sampling:** Randomly select 1,000 to 5,000 sessions for the simulation to accommodate Python's execution limits while maintaining statistical significance.

Step 3: Baseline Implementation (robustMPC Controller)

- **Prediction:** Exponential Moving Average (EMA) for bandwidth estimation.
- **Search Strategy:** Brute-force evaluation of all possible combinations for a K=5 chunk horizon (evaluating 6^5 paths per decision).
- **Objective Function:** Execute the first step of the path that maximizes: Mean Utility – (10 × Rebuffering Ratio) – Switching Rate.

Step 4: Core Implementation (SODA Controller)

- **Prediction:** Uses the exact same EMA as the baseline for a fair comparison.
- **Search Strategy (Algorithm 1):** Replaces the brute-force search with an efficient recursive search utilizing:
    - search_up(): Explores strictly increasing or flat bitrate paths.
    - search_down(): Explores strictly decreasing or flat bitrate paths.
- **Objective Function:** Executes the first step of the path that minimizes the time-based buffer penalty (Equation 1 from the SODA research paper).

Step 5: Execution Loop

For each 10-minute network trace in our sample pool, the master script will execute the following sequence:

1. Initialize Sabre with the robustMPC controller, run the trace, and save the logs.
2. Reset the Sabre simulator environment.

3. Initialize Sabre with the SODA controller, run the exact same trace, and save the logs.

Step 6: Evaluation & Visualization

- **Metrics Calculation:** The system will calculate the Logarithmic Mean Utility, Rebuffering Ratio, and Switching Rate for every session.
- **Visualization:** We will use Pandas and Matplotlib to aggregate the data and generate grouped bar charts.
- **Target Outcome:** The visual data will demonstrate that SODA maintains high video quality while significantly reducing playback stalls and aggressive bitrate switches compared to robustMPC.

Summary

In this project, we will simulate ABR streaming in a controlled, industry-standard environment. By implementing and comparing the robustMPC and SODA algorithms against real-world network traces, we will empirically show that SODA improves streaming smoothness and viewer consistency without sacrificing visual quality.

Contingency Plans

- If simulation times exceed the project timeline, we will reduce the dataset sample size for faster experimentation.
- If the recursive solver proves too complex to debug, we will simplify the dynamic programming state boundaries to ensure a working prototype.

Key Advantage

This project is highly algorithm-focused, technically meaningful, and perfectly scaled to be feasible within the available course timeline and compute resource constraints